TD SQL bonus : notions avancées

Fonctions SQL, fonctions fenêtres

2026-10-16

Avec solutions

L3 MIASHS
Université Paris Cité
Année 2025
Course Homepage
Moodle

Documentation

- pour les fonctions fenêtres : https://www.postgresql.org/docs/current/sql-expressions.html#SYNTAX-WINDOW-FUNCTIONS
- pour les fonctions SQL : https://www.postgresql.org/docs/current/sql-createfunction.html

Fonctions SQL

Forme générale

```
CREATE OR REPLACE FUNCTION  schema_name.func_name(p_arg1 p_arg1_datatype, ..., [OUT o_arg1 o_arg1_d
RETURNS some type | RECORD | SETOF some type | TABLE (..)
LANGUAGE SQL AS
$$
SQL statement ;
SQL statement ;
...
SQL statement ;
$$ ;
```

Pour toute la suite du TP, il faut remplacer `username` par votre nom de login.

Fonctions qui retournent un type simple

Ecrire une fonction qui prend en argument une région et renvoie la population maximale parmi les capitales de la région (schéma `world`).

```
CREATE OR REPLACE FUNCTION username.taille_max_capitale_region(p_region text)
RETURNS INTEGER LANGUAGE SQL AS
$$
SELECT MAX(population) AS max_pop
FROM world.country c JOIN world.city cc ON (c.capital=cc.id)
```

```
WHERE c.region=p_region AND cc.population IS NOT NULL;
$$ ;
```

Usage : quelle est la population de la capitale la plus peuplée d'Europe de l'Est ? (8389200 correspondant à Moscou)

```
SELECT *
FROM username.taille_max_capitale_region('Eastern Europe');
$$ ;
```

Fonctions qui retournent un type composé

Ecrire une fonction qui prend en argument une région et renvoie le nom et la population de la capitale la plus peuplée de cette région (schéma `world`).

On peut utiliser le *qualifiant* `OUT` pour désigner des paramètres de sortie.

```
CREATE OR REPLACE FUNCTION username.capitale(p_region text, OUT o_name_capital TEXT, OUT o_populati
RETURNS RECORD  LANGUAGE SQL AS
$$
WITH r AS(
  SELECT cc.*, RANK() OVER (PARTITION BY c.region
                            ORDER BY cc.population DESC) AS rnk
  FROM world.country c JOIN world.city cc ON (c.capital=cc.id)
  WHERE c.region=p_region)
SELECT r.name,  r.population
FROM r
WHERE r.rnk = 1 ;
$$ ;
```

Usage : quelle est le nom et la population de la capitale la plus peuplée d'Europe de l'Est ? (Moscow 8389200)

```
SELECT *
FROM username.capitale('Eastern Europe');
```

Fonctions qui retournent un type composé défini par les lignes d'une table

Ecrire une fonction qui prend en argument une région et renvoie la description de la capitale la plus peuplée de cette région (schéma `world`).

```
CREATE OR REPLACE FUNCTION username.capitale(p_region text)
RETURNS world.city  LANGUAGE SQL AS
$$
WITH r AS(
  SELECT cc.*, RANK() OVER (PARTITION BY c.region
                            ORDER BY cc.population DESC) AS rnk
  FROM world.country c JOIN world.city cc ON (c.capital=cc.id)
  WHERE c.region=p_region
)
SELECT r.id, r.name, r.countrycode, r.district, r.population
FROM r
WHERE r.rnk = 1 ;
$$ ;
```

Tester cette fonction.

Fonctions qui retournent une table

Ecrire une fonction qui prend en argument une région et renvoie la table des capitales de la région (schéma `world`).

Table de schéma explicite

On se contente d'abord de renvoyer le nom de la capitale. On explicite le schéma de la table résultat

```
CREATE OR REPLACE FUNCTION username.capitales_region(p_region text)
RETURNS TABLE (name_capital text) LANGUAGE SQL AS
$$
SELECT cc.name
FROM world.country c JOIN world.city cc ON (c.capital=cc.id)
WHERE c.region=p_region ;
$$ ;
```

Tester cette fonction.

Table de même schéma qu'une autre table

Documentation

On veut maintenant récupérer une table de même schéma que `city`.

> La solution est très simple. On profite de ce qu'à chaque table correspond un type de même nom et on utilise le mot-clé `SETOF`.

```
CREATE OR REPLACE FUNCTION username.capitales_region_large(p_region text)
RETURNS SETOF world.city  LANGUAGE SQL AS
$$
SELECT cc.*
FROM world.country c JOIN world.city cc ON (c.capital=cc.id)
WHERE c.region=p_region ;
$$ ;
```

Tester cette fonction.

Questions

1. (schéma `pagila`) Dans votre schéma personnel `username` (il faut remplacer `username` par votre nom de login), écrire une fonction SQL `film_id_cat` qui prend en paramètre une chaîne de caractère `s` et renvoie la liste des films de catégorie `s`.

> **Solution**
>
> ```
> CREATE OR REPLACE FUNCTION
>   username.film_id_cat(s text)
> RETURNS TABLE(film_id smallint) AS
> $$
>     SELECT fc.film_id
>     FROM
>         film_category fc
>   JOIN
>         category ca
>   ON (fc.category_id=ca.category_id)
>     WHERE
>     ca.name=s ;
> $$ LANGUAGE sql ;
> ```

2. (schéma `pagila`) Avec la fonction précédente, lister les acteurs qui ont déjà joué dans un film d'horreur (catégorie 'Horror').

Solution

Les solutions sont données en utilisant la fonction suivante

```
CREATE OR REPLACE FUNCTION
  entid.film_id_cat(s category.name%TYPE)
RETURNS TABLE(film_id film.film_id%TYPE)
AS $$
SELECT fc.film_id
FROM
    film_category fc
JOIN
    category ca
ON (fc.category_id=ca.category_id)
WHERE
  ca.name=s ;
$$ LANGUAGE sql;
```

Solution

Solution uniquement avec des jointures :

```
SELECT DISTINCT ac.*
FROM
  actor ac
JOIN
  film_actor fa  USING (actor_id)
JOIN
  film_id_cat('Horror') USING (film_id);
```

(156 tuples renvoyés).
Solution avec une ETC :

```
with actor_with_horror as
(
    select fa.actor_id
    from film_actor fa
    where fa.film_id in (
        select *
        from film_id_cat('Horror')
    )
)
select distinct ac.*
from actor ac join actor_with_horror using (actor_id);
```

3. (schéma `pagila`) Avec la fonction précédente, lister les acteurs qui n'ont jamais joué dans une comédie (`Comedy`) . (53 lignes)

☢ Attention ! Cette requête ne répond pas à la question :

```
SELECT DISTINCT ac.*
FROM actor ac NATURAL JOIN
     (SELECT * FROM film_actor
      WHERE film_id NOT IN
        (SELECT * FROM film_id_cat('Comedy') )
      ) as X;
```

Elle répond à la question : *Quels sont les acteurs qui ont joué dans un film qui n'est pas une comédie ?*

Solution

Réponse avec une jointure externe :

```
with actor_with_comedy as
(
    select distinct actor_id
    from film_actor fa
    join film_category fc using (film_id)
    join category c using (category_id)
    where c.name='Comedy'
)
select distinct last_name, first_name
from actor a left join actor_with_comedy ac using (actor_id)
where ac.actor_id is null
```

Une autre solution avec `EXCEPT`

```
with actor_without_comedy as
(
    (
        select distinct actor_id
        from actor
    )
    except
    (
        select distinct actor_id
        from film_actor
        join film_category using (film_id)
        join category using (category_id)
        WHERE name='Comedy'
    )
)
select last_name, first_name
from actor join actor_without_comedy using (actor_id);
```

Une autre réponse avec une sous-requête est

```
SELECT DISTINCT last_name, first_name
FROM
  actor a1
WHERE
  NOT EXISTS
  (SELECT *
  FROM
    film_actor a2
    JOIN film_category using (film_id)
    JOIN category using (category_id)
  WHERE
      a1.actor_id = a2.actor_id
      AND name='Comedy'
  );
```

ou encore (qu'en pensez-vous ?) :

Solution

```
SELECT
  DISTINCT ac.last_name, ac_first_name
FROM
  actor ac
WHERE NOT EXISTS
  (SELECT *
   FROM
    film_actor fa
   WHERE film_id IN
     (SELECT *
      FROM
        entid.film_id_cat('Comedy')
     ) AND
    fa.actor_id = ac.actor_id
  )  ;
```

Nous pouvons par exemple d'abord calculer la liste des `actor_id` des acteurs qui ont joué dans au moins une comédie, puis calculer la liste des `actor_id` des acteurs présents dans la base et faire la différence.

4. (schéma `pagila`) Avec la fonction précédente, lister les acteurs qui ont joué dans un film d'horreur ('Horror') et dans un film pour enfant ('Children'). (129 lignes)

Ici l'erreur la plus fréquente consiste à écrire

```
SELECT
  actor_id
FROM
  film_actor AS fa
WHERE
  fa.film_id IN (
    SELECT *
    FROM entid.film_id_cat('Children')
  ) AND
  fa.film_id IN (
    SELECT *
    FROM entid.film_id_cat('Horror')
  );
```

Le résultat est vide et la requête ne correspond pas à la question posée.
Elle calcule les `actor_id` des acteurs qui ont joué dans au moins un film qui relève simultanément des catégories `Horror` et `Children` (ce genre de film est assez rare).
Pour calculer un résultat correct, il faut pour chaque valeur a de `actor_id` rechercher deux tuples (pas nécessairement distincts) de `film_actor`, où l'attribut `actor_id` vaut a, et tel que, dans un cas, `film_id` désigne un film pour enfants et, dans l'autre, un film d'horreur.
En algèbre relationnelle, cela donne par exemple :

$$\pi_{\texttt{last_name},\texttt{first_name}}\Big(\texttt{actor} \bowtie \Big(\pi_{\texttt{actor_id}}(\texttt{film_actor} \bowtie \texttt{film_id_cat}(\texttt{'Children'})) \bigcap \pi_{\texttt{actor_id}}(\texttt{film_actor} \bowtie \texttt{film_id_cat}(\texttt{'Horror'}))\Big)\Big)$$

Solution

En SQL, cela peut s'écrire uniquement avec des jointures :

```
SELECT DISTINCT a.first_name, a.last_name
FROM film_id_cat('Horror') fc1
     JOIN film_actor fa1 USING(film_id)
     JOIN actor a USING(actor_id)
     JOIN film_actor fa2 ON a.actor_id = fa2.actor_id
     JOIN film_id_cat('Children') fc2 ON fa2.film_id = fc2.film_id;
```

Ou bien avec des sous-requêtes :

```
SELECT DISTINCT a.first_name, a.last_name
FROM actor a
WHERE EXISTS (SELECT film_id
              FROM film_actor AS fa1 NATURAL JOIN
                   entid.film_id_cat('Children')
              WHERE  fa1.actor_id = a.actor_id
      )
      AND
      EXISTS (SELECT film_id
              FROM film_actor AS fa2 NATURAL JOIN
                   entid.film_id_cat('Horror')
              WHERE  fa2.actor_id = a.actor_id
      ) ;
```

5. (schéma `pagila`) Ecrire une fonction `actor_category(p_nom text, p_prenom text)` qui prend en argument le nom et le prénom d'un acteur (d'une actrice) et renvoie la liste des noms des catégories de films dans lesquels il/elle a joué.

Solution

```
CREATE FUNCTION public.actor_category(p_nom text, p_prenom text)
RETURNS text
LANGUAGE SQL AS
$$
SELECT
  string_agg(DISTINCT pc.name, ', ')
FROM
  (SELECT
    actor_id
  FROM
    pagila.actor
  WHERE
    last_name = p_nom AND
    first_name = p_prenom
  ) as pa
JOIN
  pagila.film_actor as pfa USING(actor_id)
JOIN
  pagila.film_category as pfc USING(film_id)
JOIN
  pagila.category as pc USING(category_id) ;
$$ ;
```

```
bd_2023-24=# SElECT public.actor_category('LOLLOBRIGIDA', 'JOHNNY') ;
                                                  actor_category
-------------------------------------------------------------------------------------------------------------
 Action, Animation, Children, Comedy, Documentary, Drama, Games, Horror, Music, New, Sci-Fi, S
(1 row)
```

6. (schéma `pagila`) Ecrire une fonction `co_actors(p_nom text, p_prenom text)` qui renvoie les noms et prénoms des acteurs qui jouent dans un film où apparaît un acteur ou une actrice dont le nom et le prénom sont donnés en argument.

Solution

```
CREATE OR REPLACE FUNCTION public.co_actors(p_nom text, p_prenom text)
RETURNS TABLE(nom text, prenom text)
LANGUAGE SQL AS
$$
SELECT
  DISTINCT pa2.last_name as nom, pa2.first_name as prenom
FROM
  (SELECT
    actor_id
  FROM
    pagila.actor
  WHERE
    last_name = p_nom AND
    first_name = p_prenom
  ) as pa
JOIN
  pagila.film_actor as pfa1 USING(actor_id)
JOIN
  pagila.film_actor as pfa2 ON (
    pfa1.film_id=pfa2.film_id AND
    pfa1.actor_id<>pfa2.actor_id
    )
JOIN
  pagila.actor as pa2 ON (pa2.actor_id=pfa2.actor_id)
ORDER BY nom, prenom;
$$ ;
```

7. (schéma `world`) Écrire une fonction `plus_peuplee(p_countrycode text)` qui, étant donné le code d'un pays, renvoie le nom de la ville la plus peuplée de ce pays.

Solution

En remplaçant `monschema` par votre nom de schéma personnel :

```
CREATE OR REPLACE monschema.plus_peuplee(p_countrycode text)
RETURNS text
LANGUAGE sql AS $$
WITH  m AS (
  SELECT
    MAX(population) AS m_pop
  FROM
    world.city ci
  WHERE
    ci.countrycode = p_countrycode
)
SELECT
  ci.name
FROM
  world.city ci, m
WHERE ci.countrycode = p_countrycode
  AND ci.population = m_pop;
$$ ;
-- Utilisation de la fonction
SELECT monschema.plus_peuplee('FRA');
SELECT monschema.plus_peuplee('USA');
```

8. (schéma `world`) Écrire une fonction `langues_region(p_continent text)` qui étant donné le

nom d'un continent, renvoie le nombre moyen de langues parlées par pays dans chaque région. L'entête de cette fonction doit être :

```
FUNCTION langues_region(p_continent TEXT)
RETURNS TABLE(region TEXT, nbmoy NUMERIC)
```

Solution

En remplaçant `monschema` par votre nom de schéma personnel :

```
CREATE OR REPLACE FUNCTION monschema.langues_region(p_continent TEXT)
RETURNS TABLE(region TEXT, nbmoy NUMERIC)
LANGUAGE SQL AS $$
WITH S AS (
  SELECT
    R.region, cl.countrycode, COUNT(cl.language) AS n_l
  FROM
    (
      SELECT
        co.region, co.countrycode
      FROM
        world.country co
      WHERE
        co.continent = p_continent
    ) AS R
  JOIN
    world.countrylanguage cl USING (countrycode)
  GROUP BY
    R.region, cl.countrycode
)
SELECT
  region, AVG(n_l) AS nbmoy
FROM
  S
GROUP BY
  region ;
$$ ;
SELECT * FROM monschema.langues_region('Europe');
SELECT * FROM monschema.langues_region('Asia');
```

9. (schéma `world`) Écrire une requête qui liste trois attributs, pour chaque pays où on parle français, ordonné par population croissante :

- `name_country`,
- `cumul_loc` qui donne le nombre cumulé de locuteurs du français dans les pays où on parle français, dans l'ordre croissant des populations des pays,
- `cum_pop` qui donne la population cumulée des pays où on parle français, dans l'ordre croissant des populations des pays.

Utilisez une fonction fenêtre (`WINDOW`) sans partition.

Pour trouver les pays où on parle français, utilisez l'expression `language like '%French%'`. Vous remarquerez que dans certains pays, il existe plusieurs variétés de 'French'. Veillez à compter les locuteurs de toutes les formes du Français.
Que se passe-t-il si un locuteur parle plusieurs formes de Français ?

Solution

```
WITH f AS (
  SELECT
    cl.countrycode, SUM(cl.percentage) AS percentage
  FROM
    world.countrylanguage cl
  WHERE
    cl.language LIKE '%French%'
   GROUP BY
    cl.countrycode
)
SELECT
  co.name_country,
  SUM(ROUND(f.percentage * co.population_country/100)) OVER w AS cumul_loc,
  SUM(co.population_country) OVER w AS cum_pop
FROM
  f JOIN world.country co USING(countrycode)
WINDOW w AS (
  ORDER BY co.population_country
);
```

Problème : si une même personne parle plusieurs formes de Français, elle est comptabilisée autant de fois comme locuteur du Français. Mais aucune information dans la table ne permet de distinguer ce cas de figure.

Schéma `babynames`

Chaque tuple de la table `bebes` indique le `nombre` de nouveaux-nés de genre `sexe`, prénommés `prenom` mis au monde durant l'année `annee` en France.

10. (schéma `babynames`) Pour chaque année, chaque sexe, donner les dix prénoms les plus populaires et leur rang de popularité.

Solution

```
WITH R AS (
  SELECT annee, sexe, prenom, RANK() OVER(PARTITION BY (annee, sexe)
                                          ORDER BY nombre DESC) AS rnk
  FROM babynames.bebes
)
SELECT annee, sexe, prenom, rnk
FROM R
WHERE rnk <= 10 ;
```

11. (schéma `babynames`) Créer dans votre schéma personnel une vue `top_10` qui, interrogée, renvoie le résultat de la requête précédente

Solution

```
CREATE OR REPLACE VIEW babynames.top_10 AS                                    ①
  WITH R AS (
    SELECT annee, sexe, prenom, RANK() OVER(PARTITION BY (annee, sexe) ORDER BY nombre DESC) A
    FROM babynames.bebes
  )

  SELECT annee, sexe, prenom, rnk
  FROM R
  WHERE rnk <= 10
;
```

① Remplacer `babynames.top_10` par `username.top_10`.

12. (schéma `babynames`) Écrire une fonction qui prend en argument un prénom et renvoie une table de schéma `(annee, sexe, rang)` et indique le rang du prénom pour chaque année et chaque sexe.

Solution

```
CREATE OR REPLACE FUNCTION babynames.fonc_td5_1(p_prenom TEXT)
RETURNS TABLE(annee INT4, sexe TEXT, rang INT4)
LANGUAGE SQL AS
$$
WITH R AS (
  SELECT annee, sexe, prenom, RANK() OVER(PARTITION BY (annee, sexe) ORDER BY nombre DESC) AS
  FROM babynames.bebes
)
SELECT annee::int4, sexe::text, rnk::int4
FROM R
WHERE prenom = p_prenom ;
$$ ;
```

Caution
Remplacer `babynames.fonc_td5_1` par `username.fonc_td5_1`.

13. (schéma `babynames`) Pour chaque année et sexe, calculez le nombre total de naissances.

Solution

```
SELECT b.annee, b.sexe, SUM(nombre) AS nb
FROM babynames.bebes b
GROUP BY b.annee, b.sexe  ;
```

14. (schéma `babynames`) Créer dans votre schéma personnel une vue `naissances_sexe_annee` qui, interrogée, renvoie le résultat de la requête précédente

Solution

```
CREATE OR REPLACE VIEW babynames.naissances_sexe_annee AS(
    SELECT b.annee, b.sexe, SUM(nombre)  AS nb
    FROM babynames.bebes b
    GROUP BY b.annee, b.sexe
) ;
```

15. (schéma `babynames`) Pour chaque année, calculer le rapport entre le nombre total de naissances féminines et nombre total de naissances masculines.

On effectue avec les outils SQL une opération qui s'appelle un *pivot en forme élargie*.

Solution

```
WITH R AS (
 SELECT b.annee, b.sexe, SUM(nombre)  AS nb
 FROM babynames.bebes b
 GROUP BY b.annee, b.sexe
)
SELECT r1.annee, r1.nb::float4/r2.nb::float4 AS ratio
FROM R r1 JOIN R r2 ON(r1.annee=r2.annee)
WHERE r1.sexe=2 AND r2.sexe=1 ;
```

16. (schéma `babynames`) Créer dans votre schéma personnel une vue `naissances_f_h_annee` qui, interrogée, renvoie le résultat de la requête précédente.

Solution

```
CREATE OR REPLACE VIEW babynames.naissances_f_h_annee AS (
SELECT r1.annee, r1.nb AS nb_f, r2.nb AS nb_h
FROM babynames.naissances_sexe_annee r1 JOIN babynames.naissances_sexe_annee r2 ON(r1.annee=r2
WHERE r1.sexe='2' AND r2.sexe='1'
) ;
```

17. (schéma `babynames`) Pour chaque année, calculer le rapport entre le nombre total de naissances féminines et le nombre total de naissances.

Solution

```
WITH R AS (
 SELECT b.annee, b.sexe, SUM(nombre)  AS nb
 FROM babynames.bebes b
 GROUP BY b.annee, b.sexe
)
SELECT r1.annee, r1.nb::float4/(r1.nb::float4 + r2.nb::float4) AS ratio    (1)
FROM R r1 JOIN R r2 ON(r1.annee=r2.annee)
WHERE r1.sexe='2' AND r2.sexe='1' ;
```

(1) Pourquoi est-il utile de réaliser une conversion de type sur `nb` avec `nb::float4` ?

18. (schéma `babynames`) Créer dans votre schéma personnel une vue `naissances_sexe_annee` qui, interrogée, renvoie le résultat de la requête précédente.

Solution

```
CREATE OR REPLACE VIEW babynames.naissances_sexe_annee AS (
    SELECT b.annee, b.sexe, SUM(nombre)  AS nb
    FROM babynames.bebes b
    GROUP BY b.annee, b.sexe
) ;
```

19. (schéma `babynames`) Écrire une fonction qui prend en argument une année `yyyy` et renvoie une table de schéma `(annee, sexe, prenom, prop)` où `prenom` est un des dix prénoms les plus populaires donné en l'aneee `yyyy` et calculer la proportion de nouveaux-nés portant ce prénom année par année.

Tenez compte du fait que certains noms peuvent avoir une occurrence nulle certaines années.

Solution

```
CREATE OR REPLACE FUNCTION babynames.fonc_td5_2(p_annee int4)
RETURNS TABLE(annee int4, sexe TEXT, prenom TEXT, prop float4) LANGUAGE SQL AS
$$
WITH r AS (
  SELECT sexe, prenom
  FROM babynames.top_10 t
  WHERE t.annee=p_annee
), s AS (
  SELECT b.annee, r.sexe, r.prenom, b.nombre
  FROM r  LEFT OUTER JOIN babynames.bebes b
  ON(r.sexe=b.sexe AND r.prenom=b.prenom)
)
SELECT s.annee, s.sexe, s.prenom, s.nombre::float4/nsa.nb::float4 AS prop
FROM s JOIN babynames.naissances_sexe_annee nsa
ON (s.annee=nsa.annee  AND  s.sexe=nsa.sexe) ;
$$ ;
```